

You are a code translator. Please translate the following correct Python code into Rust, ensuring the same functionality and correctness. Keep the structure idiomatic for Rust (use Vec, Result, Option, etc. appropriately). Do not simplify or alter logic. Here is the Python code:

python

折叠 运行 复制

```
class Solution:
    def numberOfSpecialChars(self, word: str) -> int:
        c=0
        s=set()
        for i in range(len(word)):
            if word[i].islower():
                if word[i].lower() not in s and word[i].upper() not in s:
                    if word[i].lower() not in word[i+1:] and word[i].upper() in word:
                        s.add(word[i].lower())
                        s.add(word[i].upper())
                        c+=1
        return c
```

## HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

折叠 复制

```
use std::collections::HashSet;

impl Solution {
    pub fn number_of_special_chars(word: String) -> i32 {
        let mut count = 0;
        let mut set = HashSet::new();
        let chars: Vec<char> = word.chars().collect();

        for i in 0..chars.len() {
            if chars[i].is_lowercase() {
                let lower = chars[i].to_lowercase().next().unwrap();
                let upper = chars[i].to_uppercase().next().unwrap();

                if !set.contains(&lower) && !set.contains(&upper) {
                    let remaining_chars: String = chars[i+1..].iter().collect();
```

```

let preceding_chars: String = chars[..i].iter().collect();

if !remaining_chars.contains(lower) &&
    remaining_chars.contains(upper) &&
    !preceding_chars.contains(upper) {
    set.insert(lower);
    set.insert(upper);
    count += 1;
}
}
}
count
}
}
}

```

the previous code you provided is too time complex, please modify it ensuring the same functionality and correctness.

## HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

◇ 折叠复制

```
use std::collections::HashSet;

impl Solution {
    pub fn number_of_special_chars(word: String) -> i32 {
        let mut count = 0;
        let mut set = HashSet::new();
        let chars: Vec<char> = word.chars().collect();
        let mut upper_indices = vec![usize::MAX; 26];
        let mut lower_indices = vec![usize::MAX; 26];

        for (i, &c) in chars.iter().enumerate() {
            if c.is_lowercase() {
                let idx = (c as u8 - b'a') as usize;
                if lower_indices[idx] == usize::MAX {
                    count += 1;
                    lower_indices[idx] = i;
                }
            }
        }
    }
}
```

```

        lower_indices[idx] = 1;
    }
} else {
    let idx = (c as u8 - b'A') as usize;
    if upper_indices[idx] == usize::MAX {
        upper_indices[idx] = i;
    }
}
}

for i in 0..26 {
    if lower_indices[i] != usize::MAX && upper_indices[i] != usize::MAX {
        if lower_indices[i] < upper_indices[i] {
            let mut valid = true;
            for j in lower_indices[i]+1..upper_indices[i] {
                if chars[j].is_lowercase() && (chars[j] as u8 - b'a') as usize == i {
                    valid = false;
                    break;
                }
            }
            if valid {
                count += 1;
            }
        }
    }
}

count
}
}

```

Line 6: Char 13: error: type annotations needed for `HashSet<_>` (solution.rs) | 6 | let mut set = HashSet::new(); | ^^^^^^^^^^ ----- type must be known at this point | help: consider giving `set` an explicit type, where the type for type parameter `T` is specified | 6 | let mut set: HashSet<T> = HashSet::new(); | ++++++ For more information about this error, try `rustc --explain E0282` . error: could not compile `prog` (bin "prog") due to 1 previous error

rust

```
use std::collections::HashSet;

impl Solution {
    pub fn number_of_special_chars(word: String) -> i32 {
        let mut count = 0;
        let mut set: HashSet<char> = HashSet::new();
        let chars: Vec<char> = word.chars().collect();
        let mut upper_indices = vec![usize::MAX; 26];
        let mut lower_indices = vec![usize::MAX; 26];

        for (i, &c) in chars.iter().enumerate() {
            if c.is_lowercase() {
                let idx = (c as u8 - b'a') as usize;
                if lower_indices[idx] == usize::MAX {
                    lower_indices[idx] = i;
                }
            } else {
                let idx = (c as u8 - b'A') as usize;
                if upper_indices[idx] == usize::MAX {
                    upper_indices[idx] = i;
                }
            }
        }

        for i in 0..26 {
            if lower_indices[i] != usize::MAX && upper_indices[i] != usize::MAX {
                if lower_indices[i] < upper_indices[i] {
                    let mut valid = true;
                    for j in lower_indices[i]+1..upper_indices[i] {
                        if chars[j].is_lowercase() && (chars[j] as u8 - b'a') as usize == i {
                            valid = false;
                            break;
                        }
                    }
                    if valid {
                        count += 1;
                    }
                }
            }
        }
    }
}
```

```

    }
}

count
}
}

```

the code is not correct with the original python

## HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

折叠复制

```

use std::collections::HashSet;

impl Solution {
    pub fn number_of_special_chars(word: String) -> i32 {
        let mut count = 0;
        let mut seen = HashSet::new();
        let chars: Vec<char> = word.chars().collect();

        for i in 0..chars.len() {
            let c = chars[i];
            if c.is_lowercase() {
                let lower = c;
                let upper = c.to_ascii_uppercase();

                if !seen.contains(&lower) && !seen.contains(&upper) {
                    let mut has_upper_after = false;
                    let mut has_lower_after = false;

                    for j in i+1..chars.len() {
                        if chars[j] == upper {
                            has_upper_after = true;
                        }

                        if chars[j] == lower {
                            has_lower_after = true;
                        }
                    }

                    if !has_upper_after && !has_lower_after {
                        count += 1;
                    }
                }
            }
        }

        count
    }
}

```

```

        has_lower_after = true;
    }
}

let mut has_upper_before = false;
for j in 0..i {
    if chars[j] == upper {
        has_upper_before = true;
    }
}

if !has_lower_after && has_upper_after && !has_upper_before {
    seen.insert(lower);
    seen.insert(upper);
    count += 1;
}
}
}
}

count
}
}

```

Please optimized the code by replacing nested loops with two linear passes: first recording each lowercase letter's last occurrence and each uppercase letter's first occurrence, then checking conditions in constant time using these precomputed positions to eliminate  $O(n^2)$  scanning.

## HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

折叠复制

```

use std::collections::HashSet;

impl Solution {
    pub fn number_of_special_chars(word: String) -> i32 {
        let mut count = 0;
    }
}

```

```

let mut seen = HashSet::new();
let chars: Vec<char> = word.chars().collect();
let mut last_lower = [usize::MAX; 26];
let mut first_upper = [usize::MAX; 26];

// First pass: record last occurrence of lowercase and first occurrence of uppercase
for (i, &c) in chars.iter().enumerate() {
    if c.is_lowercase() {
        let idx = (c as u8 - b'a') as usize;
        last_lower[idx] = i;
    } else {
        let idx = (c as u8 - b'A') as usize;
        if first_upper[idx] == usize::MAX {
            first_upper[idx] = i;
        }
    }
}

// Second pass: check conditions
for i in 0..26 {
    if last_lower[i] != usize::MAX && first_upper[i] != usize::MAX {
        let lower = (b'a' + i as u8) as char;
        let upper = (b'A' + i as u8) as char;

        if !seen.contains(&lower) && !seen.contains(&upper) {
            // Check if all occurrences of lowercase are before first uppercase
            if last_lower[i] < first_upper[i] {
                seen.insert(lower);
                seen.insert(upper);
                count += 1;
            }
        }
    }
}

count
}

```

